Master Document 1: Project Foundation, Scope & SE Methodology

Project Name: Agri-Grid: Smart Agriculture & Food Security Ecosystem
Project Lead: Ameer Muawiyah (Roll No: 1512)
Context: 3rd-Semester Software Engineering, Bachelor of Science in Computer Science
Team Size: 7 Members

1. Academic Context & Project Constraints

This project is developed as a core academic deliverable for a 3rd-semester Software Engineering curriculum. The primary constraint, as dictated by course requirements, is the strict rejection of the traditional, plan-driven Waterfall model. The project demands a modern engineering approach that accommodates changing requirements, continuous testing, and the parallel development efforts of a 7-person team.

The end goal is to deliver both a fully documented Software Requirements Specification (SRS) along with unified system architecture blueprints, and a functional, integrated software prototype that demonstrates complex data handling and role-based routing.

2. Domain Overview: The "Agri-Grid" Solution

Agri-Grid is a distributed, networked, and data-intensive web application designed to modernize and secure the agricultural supply chain. It acts as a centralized digital ecosystem that replaces fragmented, manual, and undocumented farming processes.

By connecting local farming operations directly with B2B wholesale markets, logistics networks, remote agricultural experts, and compliance auditors, Agri-Grid establishes a transparent "Farm-to-Table" pipeline. The system is engineered to provide end-to-end traceability, ensuring food safety and provenance while optimizing resource allocation through predictive analytics and external environmental data integrations.

3. The Core Problem Statement

The traditional agricultural supply chain suffers from severe bottlenecks and systemic failures that this software aims to resolve. The architecture of Agri-Grid is specifically designed to address four critical industry problems:

- **Information Asymmetry (The Middleman Problem):** Farmers lack direct digital access to wholesale buyers, forcing them to rely on predatory middlemen. This results in reduced profitability for the farmer and artificially inflated prices for the buyer.
- **Logistical Inefficiencies (Cold-Chain Failures):** Without real-time transit tracking and IoT telemetry, perishable agricultural goods frequently suffer from cold-chain failures (temperature spikes) during transport, leading to massive food spoilage and disputed liability between buyers and drivers.
- **Delayed Diagnostics (Crop Disease Spread):** When crops exhibit signs of disease, farmers cannot easily access remote agronomists for timely interventions. This delay allows localized fungal or pest infections to wipe out entire harvest yields before a

treatment plan can be formulated.

- **Opaque Provenance (Lack of Traceability):** Public consumers and government compliance officers have no reliable, verifiable method to trace the origin, chemical usage, environmental impact, and safety journey of the food supply.

4. Software Engineering Methodology

To cope with the inevitable changing requirements of a complex, multi-user system, Agri-Grid employs a modern **Agile-Incremental Hybrid Model**, heavily influenced by the testing rigor of the **V-Model**.

4.1. The Agile-Incremental Hybrid Approach

The ecosystem is divided into seven distinct, manageable modules (e.g., Farmer Dashboard, B2B Marketplace, Logistics Tracker). Instead of building the entire system at once, development follows an incremental cycle:

- **Multi-Waterfall Iterations:** Each module passes through its own rapid cycle of requirements gathering, design, implementation, and testing.
- **Early Value Delivery:** This ensures that working software (e.g., a functional database and Farmer UI) is produced early in the software life cycle, lowering initial delivery risks and making it significantly easier to isolate, test, and debug smaller codebases before they are merged.
- **Agile Adaptability:** Planning remains iterative. As the team builds the B2B marketplace, they can easily pivot and adapt the database schema based on continuous feedback, without being locked into a rigid initial plan.

4.2. Rejection of the Traditional Waterfall Model

The traditional Waterfall methodology assumes that all requirements can be perfectly gathered before a single line of code is written. For a 7-person university project building a highly interconnected application, this is a dangerous anti-pattern. If the database is locked in Phase 1, any necessary changes discovered during Phase 4 (Implementation) would cause the entire project timeline to collapse. The iterative approach mitigates this exact risk.

4.3. Continuous Verification & Validation (V-Model Influence)

To ensure system stability, the team integrates the core philosophy of the V-Model: **Parallel Testing.**

- **Early Defect Detection:** Test cases are not left as an afterthought for the end of the semester. As the Requirements Engineer writes the SRS document, the QA Lead simultaneously writes the exact test cases that will validate those requirements.
- **Continuous V&V:** Every time a new feature is coded, it is immediately subjected to Verification (checking if the backend logic works via JUnit) and Validation (checking if it meets the user's actual needs via UI tests like Selenium). This guarantees that bugs are caught at the module level before they infect the global master branch on GitHub.